

Joseph Flammia

Digital Forensics – Final Project CTF Write Up

<https://github.com/stuxnet999/MemLabs/tree/master/Lab%205>

To start off, I gathered all the tools required to complete this challenge. I opted to use Kali Linux and Volatility to complete the CTF.

The first thing I did was try to find a valid profile to use for the memory dump using the following command “python2 vol.py -f MemoryDump_Lab5.raw imageinfo”. I was able to find the suggested profile “Win7SP1X64”.

```
(root@kali) ~/volatility
python2 vol.py -f MemoryDump_Lab5.raw imageinfo
Volatility Foundation Volatility Framework 2.6.1
** Failed to import volatility.plugins.registry.shutdown (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.timeliner (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
** Failed to import volatility.plugins.malware.servicediff (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.userassist (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.getsids (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.evtxlogs (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.tcaudit (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.dumpregistry (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.lsadump (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
** Failed to import volatility.plugins.malware.apihooks_kernel (ImportError: No module named distorm3)
** Failed to import volatility.plugins.registry.amcache (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.malware.check_syscall_shadow (ImportError: No module named distorm3)
** Failed to import volatility.plugins.malware.svcscan (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.auditpol (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.ssdts (NameError: name 'distorm3' is not defined)
** Failed to import volatility.plugins.registry.registryapi (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.malware.apihooks (ImportError: No module named distorm3)
** Failed to import volatility.plugins.envvars (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
INFO : volatility.debug : Determining profile based on KDBG search ...
Suggested Profile(s) : Win7SP1x64, Win7SP0x64, Win2008R2SP1x64_24000, Win2008R2SP1x64_23418, Win2008R2SP1x64, Win7SP1x64_24000, Win7SP1x64_23418
AS Layer1 : WindowsAMD64PagedMemory (Kernel AS)
AS Layer2 : FileAddressSpace (/root/.volatility/MemoryDump_Lab5.raw)
PAE type : No PAE
DTB : 0x187000L
KDBG : 0xf800028460a0L
Number of Processors : 1
Image Type (Service Pack) : 1
KPCR for CPU 0 : 0xfffff80002847d00L
KUSER_SHARED_DATA : 0xfffff78000000000L
Image date and time : 2019-12-20 03:47:57 UTC+0000
Image local date and time : 2019-12-20 09:17:57 +0530
```

Next, I ran the command “vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 pslist” to examine any processes that were active while the system was running. There were

several processes listed, but the most notable ones I found were “notepad.exe” and “WinRAR.exe”.

```
(root@kali)-[~/volatility]
# python2 vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 pslist
```

0xfffffa8000f97a20	WinRAR.exe	2924	1580	6	210	2	0	2019-12-20 03:47:13 UTC+0000
0xfffffa80010b8060	notepad.exe	2744	1580	1	57	2	0	2019-12-20 03:47:21 UTC+0000

To find out what these processes were used for, I used the “cmdline” plugin and the command “python2 vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 cmdline”

Right off the bat, I found that WinRAR was used for a file under the name “sw1WB3J0YW50.rar”. This matched the client's description of files with random letters and numbers.

```
*****
WinRAR.exe pid: 2924
Command line : "C:\Program Files\WinRAR\WinRAR.exe" "C:\Users\SmartNet\Documents\SW1wb3J0YW50.rar"
*****
```

To further analyze this .rar file, I used the filescan plugin with the command “vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 filescan | grep "SW1wb3J0YW50.rar"”

```
(root@kali)-[~/volatility]
# python2 vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 filescan | grep "SW1wb3J0YW50.rar"
Volatility Foundation Volatility Framework 2.6.1
0x000000003eed56f0 1 0 R-r-- \Device\HarddiskVolume2\Users\SmartNet\Documents\SW1wb3J0YW50.rar
```

Now with the physical offset of the file obtained, I did a deeper file scan using the following command: vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 dumpfiles -Q 0x000000003eed56f0 -D dump-files

The command above will dump the file into a directory I created called “dump-files”.

```
(root@kali)-[~/volatility]
# python2 vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 dumpfiles -Q 0x000000003eed56f0 -D dump-files
```

After dumping the file, I renamed it to the original file's name.

```
(root@kali)-[~/volatility/dump-files]
# ls
file.None.0×fffffa80010b44f0.dat

(root@kali)-[~/volatility/dump-files]
# mv file.None.0×fffffa80010b44f0.dat ~/volatility/dump-files/W1wb3J0YW50.rar
```

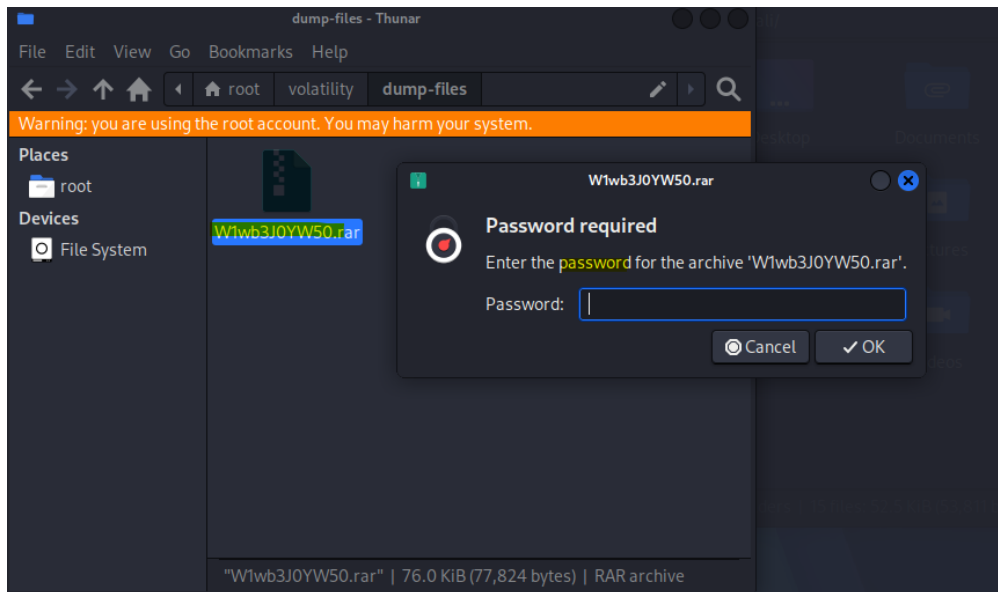
Now that the original file is intact, I attempted to view the file as shown below. A .png with the name “Stage2” is located within the .rar file.

```
# unrar l ~/volatility/dump-files/W1wb3J0YW50.rar
UNRAR 7.20 freeware      Copyright (c) 1993-2026 Alexander Roshal

Archive: /root/volatility/dump-files/W1wb3J0YW50.rar
Details: RAR 5

  Attributes      Size      Date      Time      Name
-----
* .. A....      83274    2019-12-19 07:24    Stage2.png
                        83274                        1
```

Next, I tried to extract and access the picture, but when attempting to extract I found out that the file was password protected.



The next step from here was to try and find a password for the file. I decided to use the iehistory plugin which can find browser artifacts, including URLs visited or download history.

As shown below, I was able to find a .bmp file, which is a type of picture format for a file. However, the file is encoded in base-64 format.

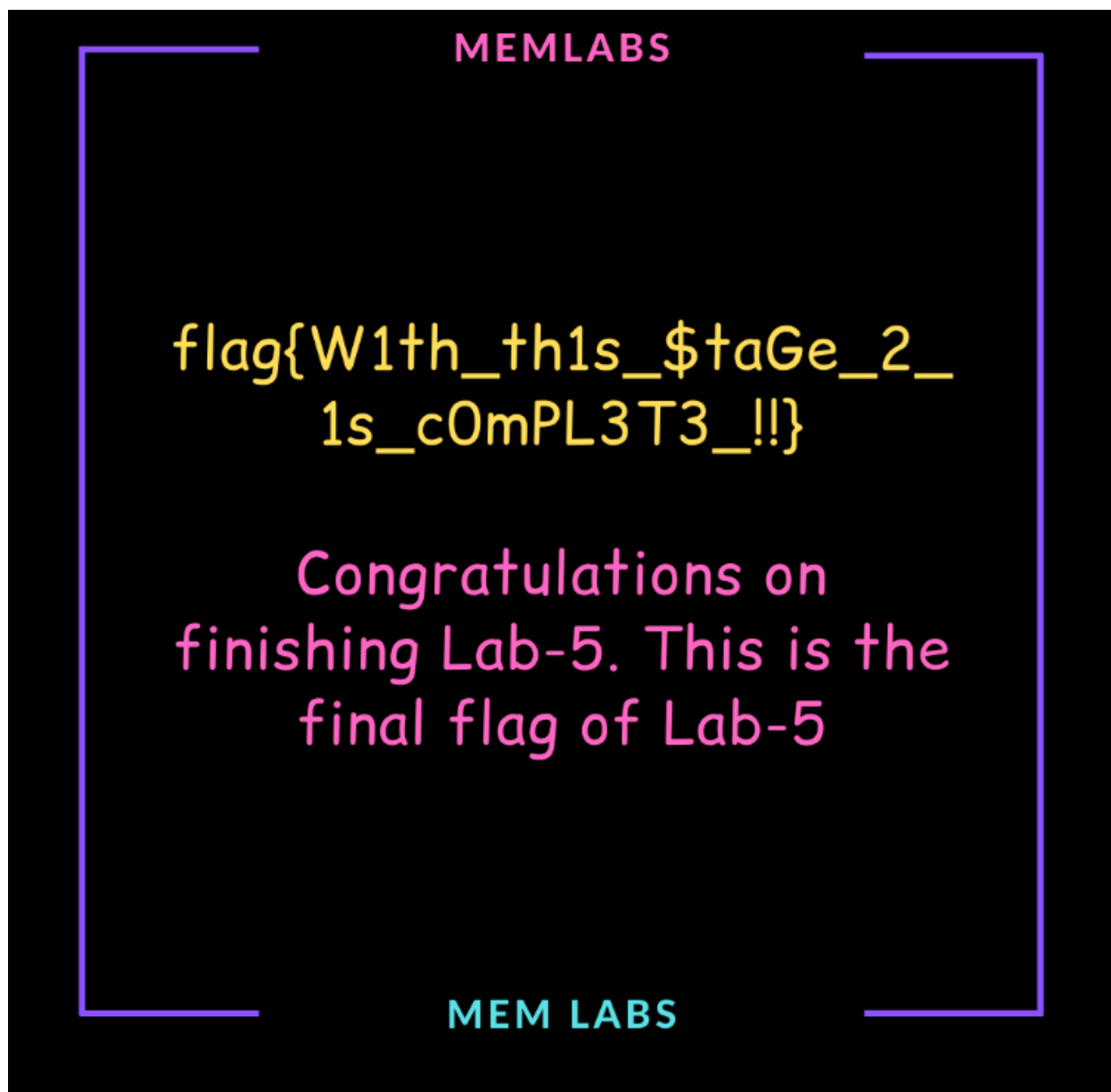
```
(root@kali) ~[~/volatility]
# python2 vol.py -f MemoryDump_Lab5.raw --profile=Win7SP1x64 iehistory | grep ".bmp"
Volatility Foundation Volatility Framework 2.6.1
Location: Visited: Alissa Simpson@file:///C:/Users/Alissa%20Simpson/Pictures/ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ.bmp
Location: Visited: Alissa Simpson@file:///C:/Users/Alissa%20Simpson/Pictures/ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ.bmp
Location: Visited: Alissa Simpson@file:///C:/Windows/AppPatch/ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ.bmp
Location: :2019122020191221: Alissa Simpson@file:///C:/Windows/AppPatch/ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ.bmp
Location: :2019122020191221: Alissa Simpson@file:///C:/Users/Alissa%20Simpson/Pictures/ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ.bmp
URL: Alissa Simpson@file:///C:/Windows/AppPatch/ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ.bmp
```

After decoding the file, I was able to find the first flag of the challenge.

```
(root@kali) ~[~/volatility]
# echo "ZmxhZ3shIV93M0xMX2QwbjNFU3Q0ZzZtMV8wZl9MNEJfM19EMG4zXyEhfQ" | base64 -d
flag{!!_w3LL_d0n3_St4g3-1_0f_L4B_3_D0n3_!!}
```

After finding the first flag, I decided to try and use this text as the password for the .rar file from before.

Entering the text reveals the image below, completing the second flag of this challenge.



There is only one flag left to find now. Going back to when I used the “cmdline” plugin, I noticed that “NOTEPAD.EXE” was in an irregular path. This would explain the “crashing” the user experienced when opening his favorite application.

```
*****
NOTEPAD.EXE pid: 2724
Command line : "C:\Users\SmartNet\Videos\NOTEPAD.EXE"
*****
```

Using the same process as before, I got the file’s physical offset and dumped the file to the dump-files directory.

```
(root@kali)-[~/volatility/dump-files]
# ls -lh ~/volatility/dump-files
total 556K
-rw-r--r-- 1 root root 68K May  3 23:12 file.None.0xfffffa8000f886e0.dat
-rw-r--r-- 1 root root 256K May  3 23:12 file.None.0xfffffa8001086b20.vacb
-rw-r--r-- 1 root root 69K May  3 23:12 file.None.0xfffffa80021a1600.img
-rw-rw-r-- 1 root root 82K Dec 19  2019 Stage2.png
-rw-r--r-- 1 root root 76K May  3 22:39 W1wb3J0YW50.rar

(root@kali)-[~/volatility/dump-files]
# mv file.None.0xfffffa8000f886e0.dat NOTEPAD.EXE

(root@kali)-[~/volatility/dump-files]
#
```

To browse the .dat file, we will have to use a decompiler, such as IDA. While IDA is not open source, it is offered for free for 30 days via a free trial.

After browsing through the internal structure of the .exe file, I was able to find this string below, which spells out final flag “bi0s{M3m_l4b5_OVeR_!”

```

add     esp, 24h
mov     eax, ds: __imp__acmdln
mov     esi, [eax]
mov     [ebp+var_20], esi
cmp     byte ptr [esi], 22h ; '"'
mov     eax, 62h ; 'b'
push    eax
mov     eax, 69h ; 'i'
push    eax
mov     eax, 30h ; '0'
push    eax
mov     eax, 73h ; 's'
push    eax
mov     ebx, 7Bh ; '{'
push    ebx
mov     ecx, 4Dh ; 'M'
push    ecx
mov     ebx, 33h ; '3'
push    ebx
mov     eax, 6Dh ; 'm'
push    eax
mov     eax, 5Fh ; '_'
push    eax
mov     eax, 6Ch ; 'l'
push    eax
mov     eax, 34h ; '4'
push    eax
mov     eax, 42h ; 'B'
push    eax
mov     eax, 35h ; '5'
push    eax
mov     eax, 5Fh ; '_'
push    eax
mov     eax, 4Fh ; 'O'
push    eax
mov     eax, 56h ; 'V'
push    eax
mov     eax, 65h ; 'e'
push    eax
mov     ebx, 52h ; 'R'
push    ebx

loc_1007527:
mov     eax, 5Fh ; '_'
push    eax
mov     eax, 21h ; '!'
push    eax
mov     eax, 7Dh ; '}'
push    eax
jnz     short near ptr loc_1007574+1

```